Automated IMPLY Logic Optimization and Mapping for Memristive Crossbar Arrays

Project 9 — Memory-Centric Computing (SS 2026), Heidelberg University

Zonghan Jia

Supervisor: Fabian Seiler

August 2026

Abstract

Material implication (IMPLY) logic computes Boolean functions inside a row of a memristive crossbar using two stateful operations, IMPLY and FALSE. Since IMPLY overwrites its destination cell, turning a logic circuit into a short, correct pulse sequence is a scheduling problem: execution order decides when a cell may be safely overwritten and reused. For this project I built a compiler that translates combinational circuits (Verilog, BLIF, or BENCH) into verified IMPLY/FALSE pulse programs for a single crossbar row. Yosys and ABC perform technology mapping against a custom IMPLY gate library. A dedicated sequencer then solves the scheduling problem in two independent layers, gate ordering and cell allocation, built on liveness tracking, an inversion cache, and a FALSE-reset packing pass that is provably minimal for a given IMPLY order. Every emitted program is checked by ABC combinational equivalence checking at four pipeline stages and by a logic-level simulation. The flow compiles a full adder to 20 steps on 7 cells (17 steps in the latency corner) and all eleven ISCAS'85 benchmarks end to end, removing 63–75 % of the naive pulse count. Compared with the SIMPLER MAGIC flow, the geometric-mean cycle overhead is 1.15× although the logic primitive is different; on `c432` my flow needs fewer cycles than the published result (218 vs. 237), and `c17` compiles to 13 steps, one below the 14-step hand-optimized IMPLY reference.

1 Introduction

Moving data between memory and processor dominates the cost of many modern workloads, and computing inside the memory array is one of the more radical answers to this "von Neumann bottleneck". Memristive crossbars support the idea at the device level: a memristor stores a bit as a resistance state, and suitable voltage pulses perform *stateful logic* directly on the stored values [1, 2]. In the IMPLY logic family, two operations suffice. A FALSE pulse resets any set of cells in a row to logic 0, and an IMPLY pulse computes material implication, $b \leftarrow \neg a \vee b$, overwriting cell b in place. Together they are functionally complete [3], so in principle any Boolean circuit can run inside one crossbar row.

In practice there is a large gap between "functionally complete" and "a working, efficient pulse program". Published IMPLY algorithms, such as the 18-step serial full adder by my supervisor [4], are hand-crafted. A human chose which intermediate values to share, which cells to sacrifice, and where one reset pulse can clear several cells at once. Project 9 asks whether this craft can be automated. The project brief names four sub-tasks: (i) a logic-level Python simulator for IMPLY logic, (ii) translation of Boolean functions into IMPLY sequences, (iii) basic optimization strategies such as step reduction and state reuse, and (iv) evaluation on a small benchmark.

This report describes the resulting tool, an end-to-end compiler from combinational circuits to verified pulse programs. Once standard synthesis tools are put to work for the Boolean part

of the problem, what remains is captured by one sentence: IMPLY *destroys its destination cell, so execution order decides when a cell can be safely overwritten and reused.* The compiler is organized around that constraint. My main contributions are:

- a complete toolflow (Section 3), in which Yosys lowers the input circuit, ABC maps it onto a custom IMPLY gate library, and a lowering pass leaves a dependency graph containing only the two hardware primitives;
- a scheduling engine (Section 4) split into two independent layers: a gate-ordering layer that races three candidate orderings, and a cell-allocation layer built on liveness counting, three safety conditions for cell reclamation, and an inversion cache that exploits the symmetry $z = \neg x \Leftrightarrow x = \neg z$;
- provably minimal reset packing (Section 4.3): grouping independent FALSE operations is reduced to the minimum piercing points problem and solved optimally for a fixed IMPLY order. This includes a negative result about where the optimality does *not* help, and the cell-reuse trade-off that does;
- a verification harness (Section 5) in which every compiled program passes four formal equivalence checks (ABC `cec`) plus a pulse-level simulation, so aggressive scheduling can never silently change the computed function;
- an evaluation (Section 6) on common circuits and the full ISCAS'85 suite [5], with a like-for-like comparison against the SIMPLER MAGIC flow [6].

2 Background

2.1 One row, two instructions

The hardware model throughout this project is a single row of n memristive cells. Each cell stores one bit: logic 1 is the low resistance state, logic 0 the high resistance state. There is no separate register file. Inputs, intermediate values, and outputs all occupy cells of the same row, so the number of cells is a real cost alongside the number of pulses.

Two operations are available:

$$\begin{aligned} \text{FALSE}(S): &\quad c_i \leftarrow 0 \ \ \forall i \in S && \text{(one pulse, any set of cells)} \\ \text{IMPLY}(a,b): &\quad c_b \leftarrow \neg c_a \vee c_b && \text{(one pulse, overwrites } c_b\text{)} \end{aligned}$$

A multi-cell FALSE counts as a single step because the reset voltage can be applied to many cells in parallel [3]. An IMPLY into a freshly cleared cell computes negation, $\text{IMPLY}(x, 0) = \neg x$, which is how inverters are realized. The pair $\{\text{IMPLY}, \text{FALSE}\}$ is functionally complete [2].

The cost model follows the literature: *latency* is the total number of pulses (steps) and *area* is the number of row cells the program touches.

2.2 The destructive-write constraint

CMOS synthesis can fan a signal out to any number of gate inputs for free. IMPLY cannot: the second operand is also the destination, so executing a gate may destroy a value that is still needed later. Copying a value first costs two extra pulses (FALSE then IMPLY), which makes naive protection expensive. The cheaper alternative, used throughout my scheduler, is to keep the *inverted* value. Producing $\neg x$ in a work cell means the original cell still holds $x = \neg(\neg x)$, and both polarities stay available without any copy. Deciding when to overwrite, when to invert, and when a cell's value is dead is the scheduling problem this project automates.

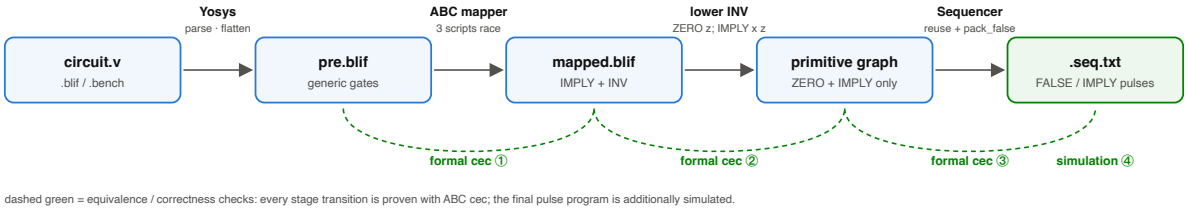

Figure 1: The compilation pipeline. Solid arrows carry the circuit through Yosys, ABC mapping, primitive lowering, and the sequencer. The dashed arcs underneath mark the four verification checkpoints (§5): three formal ABC `cec` equivalence checks between adjacent representations, plus a pulse-level simulation of the final program. To read the figure, follow the top row for what is computed and the bottom arcs for how each stage's output is proved equal to its input.

2.3 Related work

Hand-optimized IMPLY algorithms show what careful scheduling can achieve on fixed circuits; the serial full adder of Seiler et al. [4] needs 18 steps on 6 memristors and is the reference point for my full-adder results. Among automated flows, the closest work is SIMPLER MAGIC [6], which synthesizes arbitrary circuits to MAGIC [7] NOR gates using ABC with a modified cell library and maps them onto a single crossbar row with cell reuse. My flow follows the same synthesis philosophy but retargets it. The ABC library cell is the implication gate itself, and the scheduling layer handles IMPLY's destructive semantics, which NOR-based MAGIC does not have in the same form.

3 Toolflow

Figure 1 shows the pipeline. It accepts Verilog, BLIF, or BENCH input. The driver `compile.py` runs the whole chain for one circuit and prints PASS/FAIL for every check; `run_iscas85.py` batches it over a benchmark directory. Sequential circuits are rejected with a clear error, since Project 9's scope is combinational logic.

3.1 Frontend: Yosys

Yosys [8] parses and flattens the input and lowers it to generic single-bit gates in BLIF form. Nothing at this stage is IMPLY-specific; the point is only to normalize arbitrary input into a technology-independent netlist.

3.2 Technology mapping: ABC with a custom implication library

ABC [9] then maps the netlist onto a small `genlib` library whose computing cell is the implication gate $O = \neg a + b$. This is the step that turns "any circuit" into "IMPLY gates only", and it reuses two decades of logic-synthesis engineering instead of a hand-written decomposition.

One practical detail cost me some effort and is worth recording. My first library contained literally only the project primitives, CONST0 and IMPLY. ABC's standard mapper, however, expects helper cells such as inverters and buffers to exist in the library. Without them it warns, and eventually aborts with an assertion failure. The working setup is therefore an *adapter library* (`abc_imply.genlib`) that also offers INV at area 2, reflecting its true cost of two pulses, purely as a mapping aid. My supervisor confirmed this adapter approach is sound. Right after mapping, every helper is lowered away (§3.3), so no INV survives into the scheduling stage or the final program.

Different ABC scripts win on different circuits, so the compiler runs a small portfolio of three (a fast `dc2` flow, a deep `rewrite`/`refactor`/`dch` flow, and a middle variant) and keeps the

mapping with the lowest estimated cost $\#\text{IMPLY} + 2 \cdot \#\text{INV}$.

3.3 Lowering to the primitive graph

A lowering pass rewrites every helper into the two project primitives:

$$\text{INV } x \;\Rightarrow\; \text{ZERO } z;\; \text{IMPLY}(x, z),$$

and the rare constant-1 into two ZEROs and one IMPLY. Buffers (ABC occasionally emits pass-throughs) become zero-cost aliases. The result is a dependency graph containing only ZERO/IMPLY nodes. Every node is something the hardware can actually execute, with ZERO realized by membership in some FALSE pulse. On the full adder, ABC's 8 IMPLY + 4 INV mapping lowers to 12 IMPLY + 4 ZERO.

4 The sequencer

The sequencer turns the primitive graph into a linear pulse program. I structured it as two layers that do not interfere with each other:

1. *Gate ordering* decides which ready gate executes next. It changes only the sequence, never the computed function.
2. *Cell allocation* decides which physical cell holds each value. It changes only cell usage, never the legality of the order.

Neither layer can break the other's guarantee. The compiler can therefore explore orderings freely while a single allocation mechanism keeps every one of them safe. I describe allocation first because the ordering heuristics read its state.

4.1 Cell allocation: liveness, reclamation, and the inversion cache

The `remaining` counter. A pre-pass over the netlist counts, for every net, how many future gate-input uses it has. Each executed gate decrements the counter of its inputs; at zero, the value will never be read again. One subtlety here is easy to get wrong: `remaining` counts *logical* gate inputs, not physical pulses. The full adder's twelve logical gates lower into twenty pulses, but liveness is a property of the netlist, not of any particular ordering. That is why all ordering strategies can share the same counters.

Three conditions for reclaiming a cell. A cell returns to the free list only when (1) the value it holds has `remaining` $= 0$; (2) no other live value or live cached inverse (see below) sits in the same cell; and (3) the cell is not protected, i.e. not a circuit output and not an input under `--preserve-inputs`. If any condition fails the cell stays occupied. I hold this line strictly: saving a pulse is never worth risking the function.

In-place destructive writes. When a gate $\text{IMPLY}(a, b)$ is executed and b's value is at its last use, unprotected, and alone in its cell, the sequencer simply fires the pulse at b's cell: one pulse, no new cell. Otherwise it takes the constructive route via negations, $z = \neg(\neg a) \vee b$, reusing cached inverses where possible (one to five pulses).

The inversion cache. An inversion costs two pulses (FALSE then IMPLY), and inverters dominate lowered IMPLY netlists, so inversions are the largest single source of steps. The cache attacks them twice over. First, once $\neg x$ has been computed into cell z, any later use of $\neg x$ is free. Second, and this is the less obvious half, the same event also establishes $x = \neg z$: a later request for the inversion of z is answered by the cell that still holds x, at zero pulses. In the scheduler this is a pair of alias registrations. On the benchmarks it is one of the most profitable single mechanisms (§6.2). A live cached inverse keeps its cell occupied (condition 2 above), so the cache trades cells for pulses, and the ordering heuristics control how aggressively that trade is taken.

For the allocator to see inversions at all, a folding pre-pass recognizes the lowered shape ZERO z; IMPLY(x, z) (when z has a single consumer and is not an output) and treats it as an inverter for scheduling purposes only. The emitted program still contains nothing but FALSE and IMPLY.

4.2 Gate ordering: three candidates, one score function

At every point during emission, several gates may be ready. I built three candidate orderings over the same ready set:

- `topo`: a fixed topological order, blind to cell state. Simple, predictable, and a useful baseline.
- `greedy-steps`: re-scores the ready set at every step and picks the locally cheapest gate; usually the fewest pulses.
- `greedy-cells`: the same machinery with cell count as the objective; usually the fewest cells.

Both greedy modes call one score function. It combines an immediate-opportunity term s (a buffer alias scores 95, an inverter whose result is already cached scores 90 versus 10 uncached, an IMPLY whose destination is safely overwritable scores 60) with a *dying* term that counts how many of the gate's inputs reach their last use, i.e. how many cells the gate hands back. The two modes differ in exactly one constant. `greedy-steps` weights dying by 5, which makes it a tie-breaker; `greedy-cells` weights it by 50, which makes freeing cells dominate. The whole latency-versus-area personality of an ordering sits in that one number.

A concrete fork shows why watching liveness matters. Midway through the full adder two gates are simultaneously ready; both orders are legal and compute the same function. `topo` takes the netlist-order gate and finishes on 8 cells. Both greedy modes prefer the other gate, which consumes two inputs at their last use and can overwrite its destination in place, and they finish on 7 cells at the same 20 steps. One fork, one cell. A large circuit contains many such forks.

Rather than committing to one heuristic, every compilation runs all three candidates to completion and keeps the best final program, compared lexicographically by (steps, cells). No single heuristic wins on every circuit, and each candidate costs milliseconds, so there is no reason to gamble on one in advance.

4.3 Optimal FALSE packing, and an honest negative result

My supervisor's request after the ISCAS'85 milestone was to group independent FALSE operations: resets with no pending dependency should share pulses, ideally one wide reset at the start, since every merged group saves a step.

The key observation is that a reset of cell c is legal anywhere between the previous operation touching c and the next one. Each requested reset is therefore not a fixed point in the schedule

but an *interval* of legal positions in the gaps between IMPLY pulses. Covering all intervals with the fewest pulses is the classic *minimum piercing points* problem, solved optimally by the greedy sweep over earliest right endpoints. My `pack_false` pass implements this sweep, then places every chosen pulse at the earliest position its members allow. Two improvements over the previous backward-only merging fall out for free: a pulse can split per cell if only part of it is blocked, and resets can also move *later* to share a pulse with resets that come after them. The result is provably minimal for the IMPLY order the ordering layer fixed, which keeps the two layers cleanly separated.

Now the negative result, which I state plainly because I think it belongs in the record. On all eleven ISCAS'85 circuits, under all three orderings, optimal packing produced the same pulse counts as the old greedy merge. Since `pack_false` is provably minimal, the previous numbers were already optimal in this dimension. With cell reuse on, a reused cell must be reset in the gap right after its previous value dies, and no other pulse can absorb that reset. The value of the proof is that it closes a dimension: further step savings cannot come from better packing, so the effort has to go elsewhere.

That elsewhere is cell reuse itself. If a cell is never reused, its reset interval starts at position 0, so *every* reset packs into a single FALSE pulse before the first IMPLY, and

$$\text{steps} \;=\; \#\text{IMPLY} + 1.$$

This is the latency floor for a given mapping, and it motivates the first of the two scheduling corners below.

4.4 Scheduling corners: two allocation rules

Two compiler flags are easily mistaken for extra schedulers. They are not. Both are allocation rules, so they compose freely with every ordering and with each other:

- `--unlimited-cells` disables cell reuse: every value gets a fresh cell, all resets merge into one upfront FALSE, and the program sits on the $\#\text{IMPLY} + 1$ floor. This is the *min-steps* corner (SIMPLER's Table IV calls the analogous setting "UnlimitCells"). The default remains the *min-cells* corner.

- `--preserve-inputs` adds all input nets to the protected set: their cells are never overwritten and never reclaimed, so the original operands remain readable after the computation. This answers my supervisor's "save the inputs" request. In the default mode outputs land on dead input cells, which is efficient but consumes the operands.

One naming clarification I owe the reader: in *every* mode the objective the sequencer optimizes is the step count. "Min-cells" is the nickname of a *constraint* (maximal reuse), not a different objective. The default answer "20 steps" for the full adder means "the fewest steps achievable under maximal cell reuse".

5 Verification

Aggressive reuse is only defensible if it can never silently change the function, so verification is built into every single compile rather than run as an afterthought:

1. ABC `cec` at four points. Combinational equivalence is checked formally between (a) the Yosys output and the ABC mapping, (b) the mapping and the lowered primitive graph, (c) the primitive graph and the *optimized* pulse program, and (d) the primitive graph and the *naive* pulse program. For (c)/(d) the pulse program is converted back into a netlist in SSA style: one physical cell holds many values over time, so every write creates a fresh net name, otherwise `cec` would compare the wrong objects.

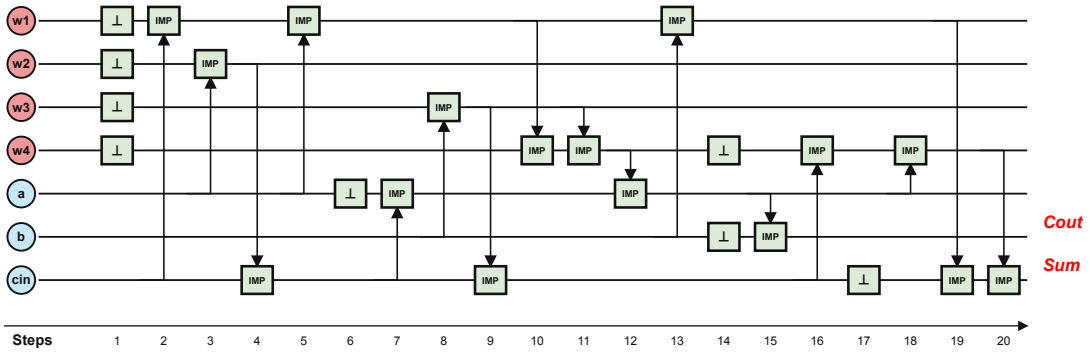

Figure 2: Compiler-generated schedule for the full adder in the default (min-cells) mode: 20 steps on 7 cells. Rows are crossbar cells (blue rails: circuit inputs; pink: work cells), columns are pulses in execution order. A ⊥ box is membership in a FALSE reset, and vertically stacked boxes share one pulse; an IMP box is an IMPLY destination, its arrow pointing at the source cell. Reading tip: watch how `sum` and `cout` end on rails that started as inputs. That reuse is why 7 cells suffice instead of 9.

2. Pulse-level simulation. The emitted program runs on the logic-level crossbar simulator from sub-task (i), `imply_sim.py`, which models one row of cells and the two pulse types, counting a multi-cell FALSE as one step. Circuits with ≤ 12 inputs are simulated exhaustively; larger ones with 1000 fixed-seed random vectors (reproducible), with formal equivalence carried by `cec`. Outputs are compared against direct netlist evaluation.

3. Unit tests. A pytest suite covers sequencer edge cases, the schedule renderer, and the benchmark runner.

If any ordering heuristic ever produced an unsafe schedule, check (c) would fail and the build would stop. Every number quoted in this report is from a run in which all five checks passed.

6 Evaluation

6.1 Worked example: the full adder

Figure 2 shows the default compilation of the 1-bit full adder: naive topological lowering costs 64 pulses, the optimized schedule 20 (16 IMPLY + 4 FALSE), on 7 cells. Three mechanisms visible in the schedule account for the difference. The 16 IMPLY pulses realize 12 graph nodes, and the 4 extra pulses are the inversions ($\neg a$, $\neg b$, and two node inverses) that the destructive-write handling requires; everything else is served in place or from the inversion cache. Eight requested resets execute in only 4 FALSE pulses thanks to `pack_false`. And all three input cells are reclaimed the moment their values die, so the outputs land on former input rails.

Table 1 walks the same circuit through the four corner configurations. `--preserve-inputs` keeps a, b, c_{in} readable to the end for three extra cells at unchanged latency. `--unlimited-cells` reaches the #IMPLY + 1 floor of 17 steps with one upfront FALSE. Both flags together give readable inputs *and* 18 steps, two below the default, which answers the supervisor's full-adder question directly. The 18-step hand-optimized adder of [4] remains the tighter design point on 6 memristors; my automated flow lands within two pulses of it (equal, in the both-flags corner) while handling arbitrary circuits with no hand-crafted knowledge. On `c17`, preserving inputs is even free: protecting the inputs steers the scheduler to a 14-step schedule, one *below* the 15-step default. A reminder that these heuristics are not monotone in their constraints. Figure 3 shows the two corner schedules.

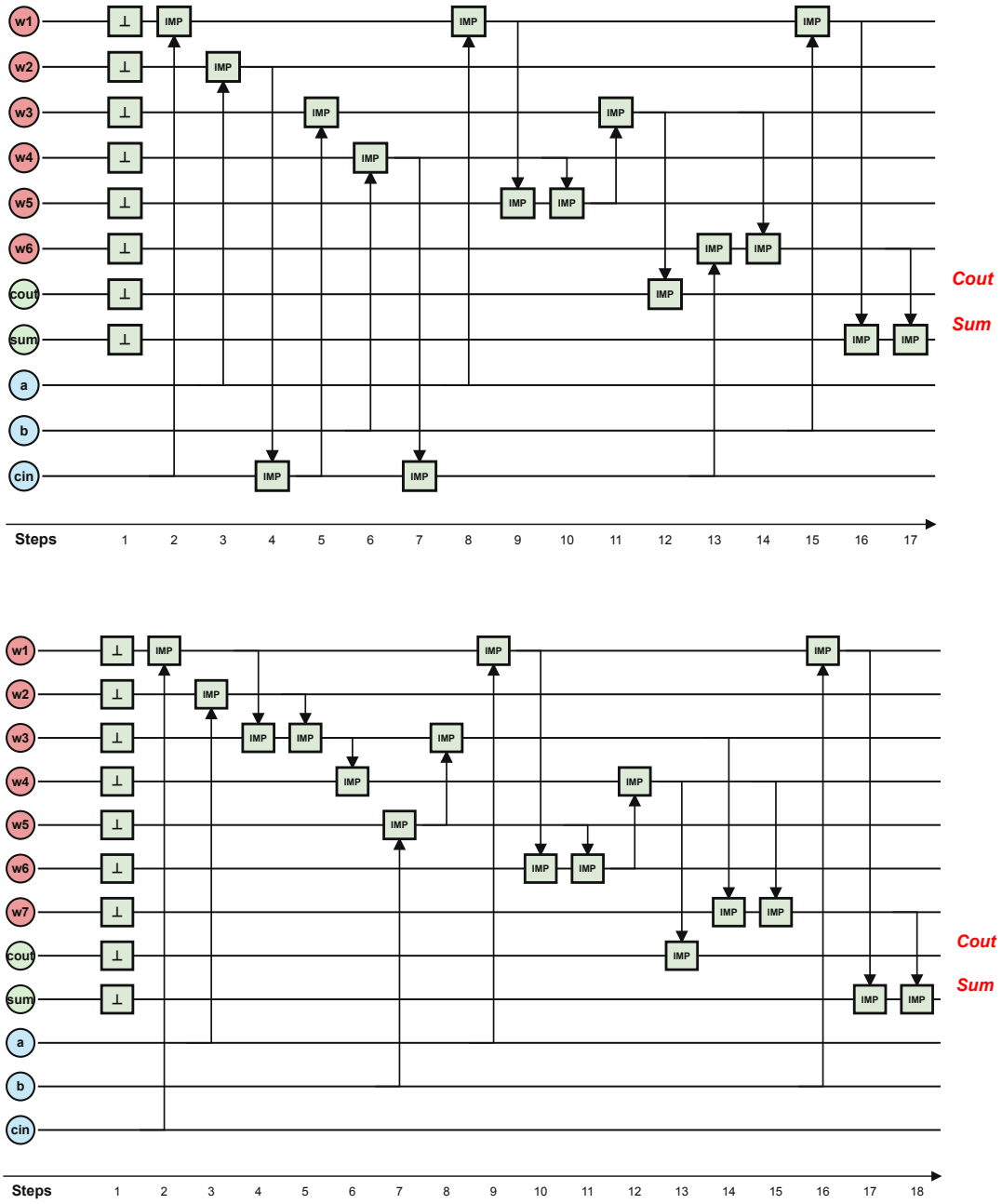

Figure 3: The full adder in the latency corner. Top: `--unlimited-cells`, 17 steps / 11 cells. Every reset has merged into the single wide FALSE pulse at column 1, after which only IMPLY pulses run. Bottom: both flags, 18 steps / 12 cells. Same upfront reset, but no box ever lands on the three input rails, so a, b, c_{in} are still readable at the end.

Table 1: Scheduling corners on the full adder and on `c17`. Every configuration passes the full verification chain. Steps count all pulses; cells count all row cells touched, inputs included.

	full adder		`c17`	
mode	steps	cells	steps	cells
default (min-cells)	20	7	15	8
`--preserve-inputs`	20	10	14	10
`--unlimited-cells`	**17**	11	**13**	11
both flags	**18**	12	**13**	11

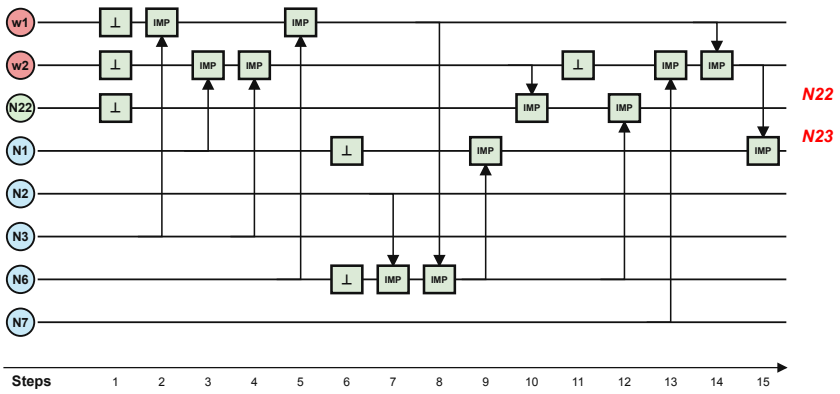

Figure 4: Generated schedule for `c17` (default mode, 15 steps / 8 cells). Six requested resets execute as three FALSE pulses; the cells of inputs `N1` and `N6` are recycled at step 6, the moment their values die.

6.2 Generality: `c17` and the ISCAS'85 suite

Nothing in the flow is full-adder-specific; the same two commands compile every circuit in the repository. The smallest ISCAS'85 circuit, `c17` (six NAND gates), maps to 6 IMPLY + 4 INV, lowers to 10 IMPLY + 4 ZERO, and compiles to 15 steps / 8 cells (Figure 4). NAND is a friendly target because $\neg(x \wedge y) = \text{IMPLY}(x, \neg y)$. In the min-steps corner it needs 13 steps, one below the 14-step hand-optimized IMPLY implementation quoted to me as the state-of-the-art reference for this circuit.

Table 2 reports the whole suite. All eleven circuits compile and verify end to end, in both corners, and scheduling alone removes 63–75 % of the naive pulse count. The min-steps corner saves a further 8–19 % latency over min-cells at the price of roughly 2–2.6× the cells. `c432`, for example, drops from 243 to 218 steps while growing from 72 to 132 cells.

6.3 Comparison with SIMPLER MAGIC

For the comparison in Table 3 to mean anything, the accounting has to be stated first. Both flows compute one circuit inside a single crossbar row. Both count compute pulses plus reset pulses, and a multi-cell reset is one cycle on either side. Neither counts initial input loading, and both cell figures include the inputs. The netlists, however, differ by construction: SIMPLER maps to NOR2 gates, I map to IMPLY gates via ABC with a custom library, so per-circuit gate counts, and therefore cycle floors, are not identical.

Overall the flow is competitive with the published state of the art. The geometric mean of the cycle ratio is 1.15× in the min-steps corner (1.36× in min-cells), with a different logic primitive, an automated flow, and formal verification on every row. On `c432` my flow wins

Table 2: ISCAS'85, default (min-cells) corner: naive topological emission versus the optimized schedule. All rows PASS the full verification chain. The rightmost column is the fraction of naive pulses removed by scheduling alone; the mapping is identical in both columns.

circuit	naive steps	opt. steps	cells	saved
c17	54	15	8	72 %
c432	971	243	72	75 %
c499	2901	885	228	69 %
c880	2100	631	160	70 %
c1355	2732	888	229	67 %
c1908	2621	807	208	69 %
c2670	3914	1235	367	68 %
c3540	6188	1981	323	68 %
c5315	9005	2867	640	68 %
c6288	12422	4616	741	63 %
c7552	10014	3190	717	68 %

Table 3: My flow versus SIMPLER MAGIC [6] (their Table II) on ISCAS'85. SIMPLER numbers are as printed in the paper; c17 is not reported there. The last column is the better of my two corners divided by SIMPLER's #Cyc; values below 1 mean my flow needs fewer cycles.

	SIMPLER		ours (steps / cells)		
circuit	#Cyc	#Mem	min-cells	min-steps	best ratio
c432	237	62	243 / 72	**218** / 132	**0.92**
c499	620	110	885 / 228	761 / 414	1.23
c880	512	142	631 / 160	543 / 296	1.06
c1355	619	111	888 / 229	766 / 418	1.24
c1908	588	122	807 / 208	684 / 361	1.16
c2670	891	383	1235 / **367**	1004 / 704	1.13
c3540	1434	192	1981 / 323	1684 / 811	1.17
c5315	2002	351	2867 / 640	2439 / 1301	1.22
c6288	2938	149	4616 / 741	3731 / 1892	1.27
c7552	2227	535	3190 / 717	2630 / 1450	1.18

outright (218 vs. 237 cycles, −8 %), and c880 is within 6 %. In the min-cells corner c2670 uses fewer cells than SIMPLER (367 vs. 383).

The remaining gap is a mapping effect, not a scheduling effect. XOR costs more IMPLY gates than NOR2 gates, and the worst ratios are exactly the XOR-heavy circuits (c499/c1355, and the c6288 multiplier). On the scheduling side there is no slack left: packing is provably minimal for a fixed order, and min-steps sits on the #IMPLY + 1 floor.

The two corners also span a real trade-off rather than a single point. Min-steps buys roughly 15 % latency for about 2.5× the cells (cell geomean vs. SIMPLER's #Mem: 1.66× in min-cells mode); SIMPLER's own Table IV sweeps the same latency/area curve with row size. I report both ends and let the application pick its point.

7 Discussion

What the negative result bought. Proving the reset packing optimal did not reduce a single benchmark number, and that is precisely its value. It closed a dimension: pulse counts in the cell-reuse regime were already at their floor, so engineering effort could move with confidence

to the levers that do pay (cell reuse corners, and next, mapping). Knowing where there is no slack is as useful as knowing where there is.

Limitations. The flow handles combinational logic only; sequential circuits are rejected at the frontend. The cost model is logic-level, in pulses and cells, and says nothing about device-level latency, energy, or non-ideality effects such as state drift over long IMPLY chains, which [4] shows matter in practice. Cell usage in the min-cells corner averages 1.66× SIMPLER's, a direct price of the inversion cache's cells-for-pulses trade. And the comparison inherits whatever differences exist between the two papers' netlist generations, as discussed above.

Future work. The analysis in §6.3 makes the next lever explicit: *XOR-aware mapping*, i.e. teaching the mapping stage cheaper IMPLY realizations of XOR-shaped structures, rather than further scheduling work. On the validation side, the natural next step is device-level cross-checking of the generated sequences with my supervisor's ATOMIC tool [10], closing the loop from compiler output to SPICE-level simulation. A Pareto mode that sweeps the row-width budget between the two corners would also turn the two reported points into the full trade-off curve.

8 Conclusion

I set out to automate what published IMPLY algorithms do by hand. One constraint drove the whole design, namely that IMPLY destroys its destination cell, and the answer is a compiler whose scheduler splits into two independent layers: an ordering layer that races three candidate heuristics sharing one score function, and an allocation layer built on liveness counting, three reclamation conditions, an inversion cache, and provably minimal reset packing. Two flags expose the latency/area corners of that machinery as allocation rules, not special cases. The flow compiles arbitrary combinational circuits: a full adder in 20 steps on 7 cells, or 17 steps in the latency corner, and all eleven ISCAS'85 benchmarks with 63–75 % of naive pulses removed, each emitted program proved equivalent to its source at four pipeline stages. Against SIMPLER MAGIC the geometric-mean cycle overhead is 1.15×, with an outright win on `c432` and a 13-step `c17` below the hand-optimized reference. The code, figures, and result tables are public, and two commands regenerate every number in this report.

Acknowledgment

I thank my supervisor Fabian Seiler for steering this project at every important fork: the suggestion to use ABC with a custom implication library instead of a hand-written decomposition, the pointer to the ISCAS'85 benchmarks and the SIMPLER comparison, and the concrete scheduling requests (exposing FALSE structure to the scheduler, storing inverted values for reuse, grouping independent resets, and keeping the original inputs readable) that shaped the optimization work reported here.

Declaration on the use of AI tools

AI assistants (Claude, Anthropic) were used in this project in four ways. First, for brainstorming: talking through design alternatives, such as scheduling heuristics and benchmark scope, before I settled on a decision. Second, for search and retrieval: locating literature and bibliographic data, and looking up Yosys and ABC usage details. Third, for explanation: having concepts from the course material and the cited papers explained to me while I worked. Fourth, for writing and implementation support: drafting and language-editing this report from my project

notes, communication records, and result tables, and assisting with debugging in the toolchain itself.

References

[1] Julien Borghetti, Gregory S. Snider, Philip J. Kuekes, J. Joshua Yang, Duncan R. Stewart, and R. Stanley Williams. 'memristive' switches enable 'stateful' logic operations via material implication. *Nature*, 464(7290):873–876, 2010.

[2] Eero Lehtonen and Mika Laiho. Stateful implication logic with memristors. In *2009 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH)*, pages 33–36, 2009.

[3] Shahar Kvatinsky, Guy Satat, Nimrod Wald, Eby G. Friedman, Avinoam Kolodny, and Uri C. Weiser. Memristor-based material implication (IMPLY) logic: Design principles and methodologies. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 22(10):2054–2066, 2014.

[4] Fabian Seiler, Gulafshan, and Nima TaheriNejad. An efficient robust serial IMPLY-based in-memristor adder. In *IEEE Cross-disciplinary Conference on Memory-Centric Computing (CCMCC)*, 2025.

[5] Franc Brglez and Hideo Fujiwara. A neutral netlist of 10 combinational benchmark circuits and a target translator in FORTRAN. In *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 677–692, 1985.

[6] Rotem Ben-Hur, Ronny Ronen, Ameer Haj-Ali, Debjyoti Bhattacharjee, Adi Eliahu, Natan Peled, and Shahar Kvatinsky. SIMPLER MAGIC: Synthesis and mapping of in-memory logic executed in a single row to improve throughput. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(10):2434–2447, 2020.

[7] Shahar Kvatinsky, Dmitry Belousov, Slavik Liman, Guy Satat, Nimrod Wald, Eby G. Friedman, Avinoam Kolodny, and Uri C. Weiser. MAGIC—memristor-aided logic. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 61(11):895–899, 2014.

[8] Clifford Wolf and Johann Glaser. Yosys — a free Verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, 2013.

[9] Robert Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification (CAV 2010)*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010.

[10] Fabian Seiler and Nima TaheriNejad. ATOMIC: Automatic tool for memristive IMPLY-based circuit-level simulation and validation. https://github.com/fabianseiler/ATOMIC, 2025.